Twenty-Second Implementation Iteration: User Type Polymorphism and UserFactory

with Cleanup of Obsolete UserRole Enum Dependencies

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this twenty-second implementation iteration is to refactor the user model so that different user types are represented as real polymorphic objects rather than as a single value object with an attached role enum.

Before this iteration, the system stored a concrete `User` object containing `userId`, `name`, and a `UserRole` enum. Requests also stored the requester by value. This design worked for earlier iterations, but it would cause object slicing if user subclasses were introduced. It also kept role-specific behavior dependent on enum-based logic rather than encapsulating that behavior inside user objects themselves.

In this iteration, `User` is refactored into an abstract base class, and four concrete user subclasses are introduced: `Student`, `Instructor`, `Staff`, and `Administrator`. A `UserFactory` is added to create the correct user subtype from CSV role strings. After the main refactor, the old enum-based role helpers were reviewed, confirmed to be unused in runtime source code, and removed as a cleanup improvement under the same iteration.

2. Changes from Iteration 21

Compared with the twenty-first iteration, the following changes were introduced:

- Refactored `User` into an abstract polymorphic base class
- Added concrete user subclasses:
  - `Student`
  - `Instructor`
  - `Staff`
  - `Administrator`

- Added `UserFactory` for creating user objects from CSV role names
- Changed requests to store `User*` instead of storing `User` by value
- Updated `SystemData` and user loading to use `vector<User*>`
- Moved priority and space authorization behavior into user subclasses
- Updated `UserRoleRule` so it delegates authorization to polymorphic user behavior
- Updated console output and CSV export to use role names from user objects
- Removed obsolete `UserRole.h` after confirming it was no longer used by runtime source code

This iteration does not change the allocation algorithms, the rule engine structure, the CSV file format, or the approval and rejection behavior of requests.

3. Scenario

The prototype starts by loading users, spaces, and requests from CSV files through the `DataController`. During user loading, the system reads the role string from `users.csv` and passes it to `UserFactory`. The factory creates the correct user subtype and returns it through the base `User` interface.

When requests are loaded from `requests.csv`, each request stores a pointer to an already created user object rather than storing a copied user value. This avoids object slicing and preserves polymorphic behavior. Once request processing begins, the existing rule engine and selected allocation strategy continue working as before. However, when role-specific decisions are required, the system now asks the user object directly for its role name, priority, and allowed space types.

At the end of execution, the prototype still exports:

- approved allocations to `allocations.csv`
- processed request outcomes to `request_results.csv`
- request lifecycle history together with the exported request results

4. Modules and Their Tasks

A. User Base Class Module

- Store the common user data: identifier and name

- Provide shared user access methods
- Define abstract behavior for role name, priority, and allowed space types

B. User Subclass Module

- Represent each user type as a separate concrete class
- Define role-specific behavior inside each subclass
- Remove the need for switch-based role logic in runtime code

C. UserFactory Module

- Create the correct user subtype from the CSV role string
- Return user objects through the base `User` interface
- Limit its responsibility to object creation only

D. Request and Data Loading Module

- Store requesters as `User*` to preserve polymorphism
- Load user collections as `vector<User*>`
- Ensure cleanup deletes requests, spaces, and users safely

E. Authorization and Reporting Module

- Keep `UserRoleRule` as the authorization rule gateway
- Delegate actual permission checks to user subclass behavior
- Use polymorphic role name and priority in console and CSV output

5. User Type Policy

The user subtype policy implemented in this iteration is as follows:

- **Student**: role name `Student`, priority 1, may request `Classroom` and `MeetingRoom`
- **Staff**: role name `Staff`, priority 2, may request `MeetingRoom`

- **Instructor**: role name `Instructor`, priority 3, may request `Classroom`, `Laboratory`, and `MeetingRoom`
- **Administrator**: role name `Administrator`, priority 4, may request all supported space types

This policy is now implemented inside the user subclasses themselves. As a result, request priority is derived from `requester->getPriority()`, and authorization is checked through `requester->canRequestSpaceType(...)`.

6. Cleanup Improvement

After the main polymorphism refactor was completed, the previous enum-based role system was reviewed. The only remaining old runtime artifact was `src/models/UserRole.h`. That file defined:

- `UserRole`
- `userRoleToString(...)`
- `stringToUserRole(...)`
- `userRoleToPriority(...)`

No live runtime source file still used those helpers or included that header. Therefore, `UserRole.h` was removed. This cleanup is important because it prevents the system from maintaining two competing role models at the same time.

After the cleanup:

- reporting uses `getRoleName()`
- priority uses `getPriority()`
- authorization uses `canRequestSpaceType(...)`

The remaining `UserRole` references are only in names such as `UserRoleRule` or in documentation, not in the old enum-based runtime system.

7. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
18	Same + strategy-based processing	Class + Lab + Meet	Allocation Strategy Pattern + Greedy strategy
19	Same + explicit greedy batch flow	Class + Lab + Meet	Greedy batch processing moved into strategy
20	Same + factory-based strategy selection	Class + Lab + Meet	Singleton strategy factory + interface-based selection
21	Same + priority-based processing	Class + Lab + Meet	Added `PriorityAllocationStrategy`
22	Same + polymorphic users	Class + Lab + Meet	User hierarchy + UserFactory + removal of obsolete enum role helpers

8. Updated Class Diagram

Figure 1 shows the updated class diagram including the new user hierarchy, the `UserFactory`, pointer-based requester references, and the removal of the old enum-based role dependency.

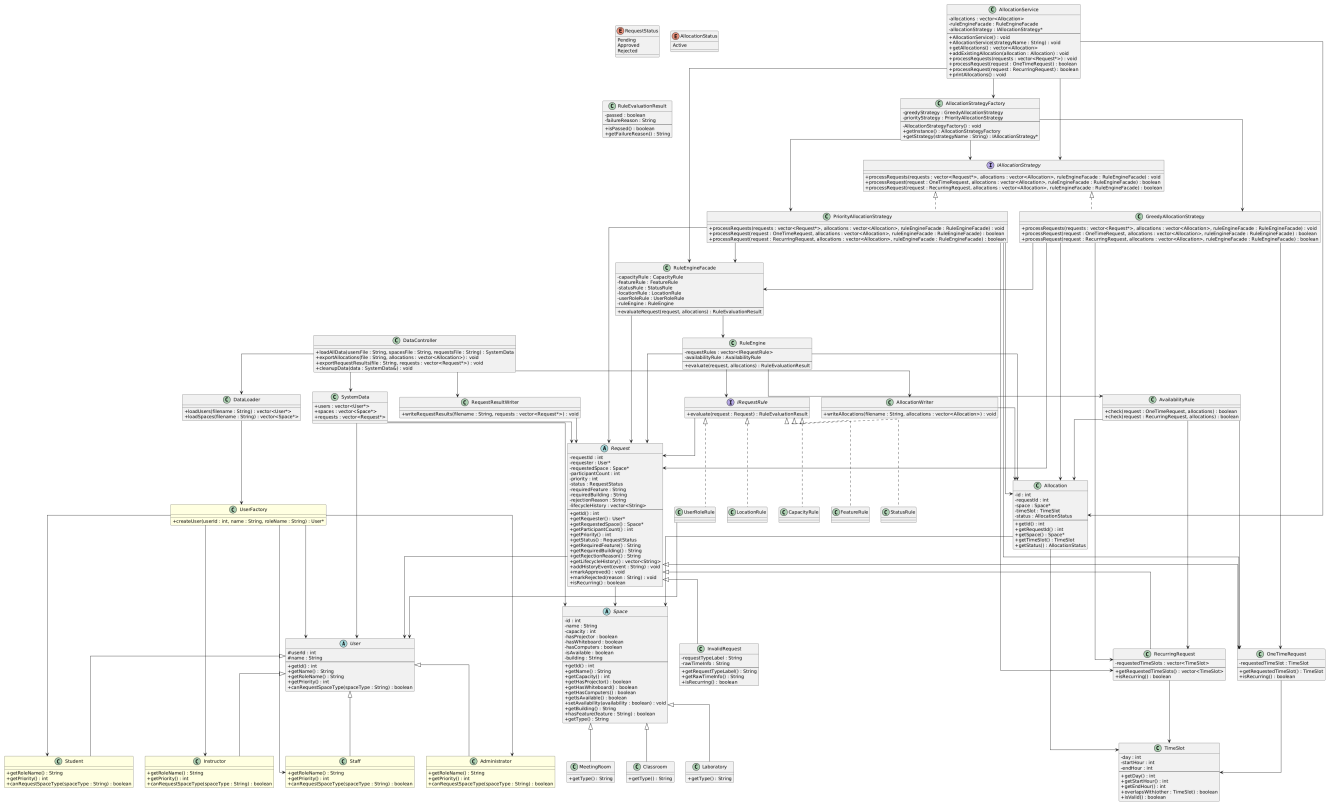

Figure 1: Updated Class Diagram with User Type Polymorphism and UserFactory

9. Updated Workflow Diagram

Figure 2 shows the updated workflow. User objects are now created through the factory, attached to requests through pointers, and queried polymorphically during authorization, reporting, and priority handling.

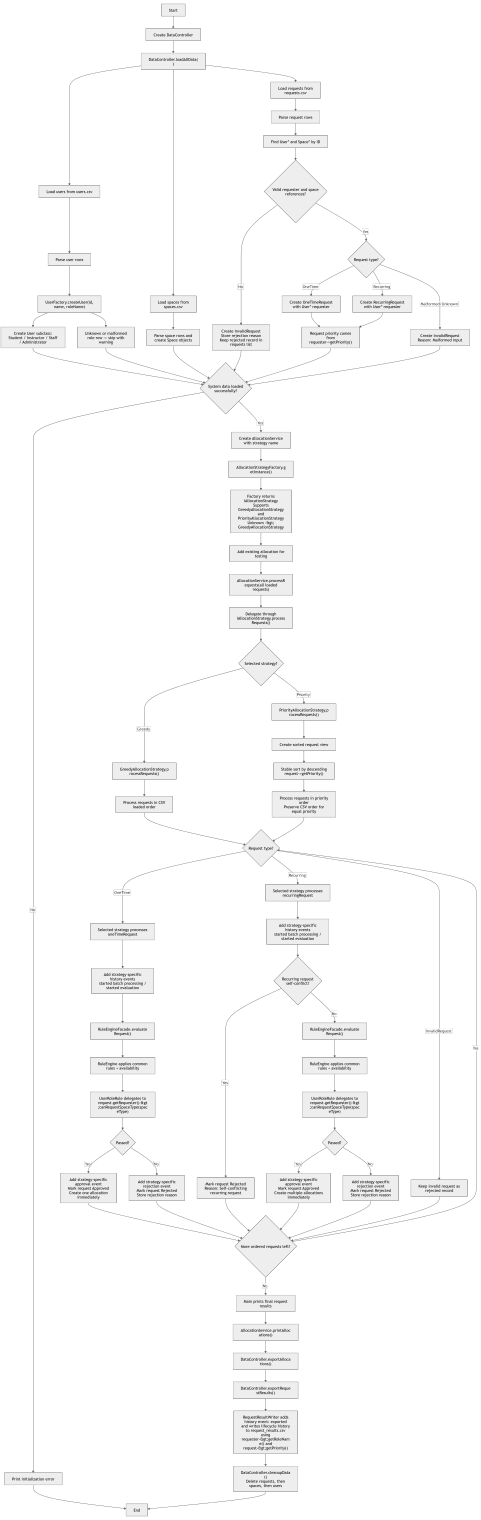

Figure 2: Workflow with Polymorphic User Creation and Usage

10. Prototype Implementation

The C++ prototype was extended by refactoring `User` into an abstract base class and adding concrete subclasses for the supported user types. Each subclass now defines its own role name, priority, and allowed space types. This change makes user-specific behavior explicit and encapsulated inside the correct object rather than relying on switches over a separate role enum.

To support safe construction from CSV input, `UserFactory` was introduced. The factory reads the role string and returns the correct user subtype through the base interface. The factory is responsible only for creating user objects. It does not apply allocation rules and does not decide whether a request should be approved.

Requests were updated to store `User*` instead of storing user objects by value. This avoids object slicing and allows the request to preserve the real subtype behavior of the requester. The data-loading structures were updated accordingly, and cleanup logic was revised so that requests, spaces, and users are deleted safely in the correct ownership layer.

The authorization flow remains centered around `UserRoleRule`, but the internal behavior of that rule is now polymorphic. Instead of switching on a role enum, it asks the requester whether the requested space type is allowed. Console output and CSV export were also updated to use the requester pointer together with `getRoleName()` and `getPriority()`.

Finally, the previous enum-based helper system was removed after verification showed that runtime source code no longer depended on it. This cleanup simplifies the design and ensures that the project maintains only one active user-role model.

11. Summary

This twenty-second implementation iteration refactors the user model using polymorphism and the Factory Pattern. Users are now represented by a base `User` interface together with concrete subclasses for `Student`, `Instructor`, `Staff`, and `Administrator`. `UserFactory` creates these user objects from CSV role strings, while requests safely reference them through pointers.

The iteration also includes an important cleanup improvement: the obsolete `UserRole.h` enum helper system was removed after confirming that runtime behavior had already been fully migrated to the polymorphic model. As a result, role reporting, priority, and authorization now all come from the user subclasses themselves, while the existing rule engine, allocation strategies, CSV formats, and export behavior remain unchanged.